

Technical Support Ticket Management System

MÓDULO 4 — CRUD Básico de Tickets (Creación)

Objetivo: Implementar el formulario de creación de tickets utilizando componentes de Blazor y validación de datos.

En este **MÓDULO 4**, se construye el primer puente entre el usuario y la base de datos. Se implementa la funcionalidad de **Crear Ticket**, lo cual nos permite ver cómo los modelos, los enums y las relaciones definidas cobran vida en una interfaz interactiva.

CRUD es un acrónimo muy utilizado en desarrollo de software y bases de datos que representa las cuatro operaciones fundamentales para gestionar datos en un sistema:

	Operación	Descripción
C	Create	Crear o insertar nuevos datos en la base de datos
R	Read	Leer o consultar datos existentes
U	Update	Actualizar o modificar datos existentes
D	Delete	Eliminar datos de la base de datos

Estas operaciones constituyen la base de prácticamente **todas las aplicaciones que trabajan con datos**, desde sistemas simples hasta plataformas empresariales.

- 1. Create (Crear):** Agregar un nuevo usuario.
`INSERT INTO Usuarios (Nombre, Email) VALUES ('Carlos', 'carlos@email.com');`
- 2. Read (Leer / Consultar):** Obtener los usuarios almacenados.
`SELECT * FROM Usuarios;`
`SELECT * FROM Usuarios WHERE Id = 1;`
- 3. Update (Actualizar):** Modificar información de un usuario.
`UPDATE Usuarios`
`SET Email = 'juan_nuevo@email.com'`
`WHERE Id = 1;`
- 4. Delete (Eliminar):** Eliminar un usuario de la tabla.
`DELETE FROM Usuarios WHERE Id = 2;`

Relación con aplicaciones modernas

En el desarrollo de aplicaciones **FullStack**, CRUD normalmente se implementa en varios niveles:

1. Base de datos

- SQL Server
- MySQL
- PostgreSQL

2. Backend

- API o servicios que ejecutan las operaciones CRUD

3. Frontend

- Formularios para:
 - Crear registros
 - Ver listas
 - Editar datos
 - Eliminar registros

Por ejemplo, en un **Sistema de Gestión de Tickets de Soporte**:

Operación	Ejemplo
Create	Crear un ticket de soporte
Read	Ver lista de tickets
Update	Cambiar estado del ticket
Delete	Eliminar un ticket cerrado

CRUD y HTTP (muy usado en APIs)

CRUD	Método HTTP
Create	POST
Read	GET
Update	PUT / PATCH
Delete	DELETE

1. Preparación: El Servicio de Datos (TicketService)

Aunque mantenemos una estructura simple, es una buena práctica mover la lógica de base de datos fuera del archivo `.razor`. Esto mantiene la interfaz limpia.

Crea la carpeta `/Services` y el archivo `TicketService.cs`. Usaremos el **Primary Constructor** de C# 14.

Archivo: `/Services/TicketService.cs`, copiar el código:

```
// Usando Primary Constructor de C# 14
6 references
public class TicketService(IDbContextFactory<ApplicationDbContext> dbFactory)
{
    // --- MÓDULO 4: MÉTODOS DE CREACIÓN ---

    1 reference
    public async Task<List<Category>> GetCategoriesAsync()
    {
        using var context = dbFactory.CreateDbContext();
        return await context.Categories.ToListAsync();
    }

    1 reference
    public async Task CreateTicketAsync(Ticket ticket)
    {
        using var context = dbFactory.CreateDbContext();
        context.Tickets.Add(ticket);
        await context.SaveChangesAsync();
    }
}
```

Registra este servicio en `Program.cs`:

```
15     builder.Services.AddCascadingAuthenticationState();
16     builder.Services.AddScoped<IdentityRedirectManager>();
17     builder.Services.AddScoped<AuthenticationStateProvider, IdentityRevalidatingAuthenticationStateProvider>();
18     builder.Services.AddScoped<TicketService>(); // Register the TicketService for dependency injection
19
```

2. Datos Iniciales (Seed Data)

Para que el formulario funcione, se necesita categorías en la base de datos. Agregar un pequeño bloque en `Program.cs` (antes de `app.Run()`) para insertar datos de prueba si la tabla está vacía. Copiar según se muestra:

```
69
70     using (var scope = app.Services.CreateScope())
71     {
72         var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
73         if (!context.Categories.Any())
74         {
75             context.Categories.AddRange(
76                 new Category { Name = "Hardware", Description = "Physical devices and components" },
77                 new Category { Name = "Software", Description = "Applications and OS issues" },
78                 new Category { Name = "Network", Description = "Connectivity and Internet" }
79             );
80             context.SaveChanges();
81         }
82     }
83
84     app.Run();
```

3. La Interfaz de Usuario: CreateTicket.razor

Crear la carpeta `/Components/Pages/Tickets` y dentro el archivo crear `CreateTicket.razor`.

Archivo: `/Components/Pages/Tickets/CreateTicket.razor`

```
1  @page "/tickets/create"
2  @using Microsoft.AspNetCore.Authorization
3  @using SupportTicketsSystem.Enums
4  @using SupportTicketsSystem.Models
5  @using SupportTicketsSystem.Services
6  @using System.Security.Claims
7  @inject TicketService TicketService
8  @inject NavigationManager Navigation
9  @inject AuthenticationStateProvider AuthProvider
10 @attribute [Authorize]
11 @rendermode InteractiveServer
12
13 <PageTitle>Create New Ticket</PageTitle>
14
15 <div class="container mt-4">
16     <h2>Create New Support Ticket</h2>
17     <hr />
18
19     <EditForm Model="newTicket" OnValidSubmit="HandleSubmit" FormName="CreateTicketForm">
20         <DataAnnotationsValidator />
21         <ValidationSummary class="text-danger" />
22
23         <div class="mb-3">
24             <label class="form-label">Title</label>
25             <InputText @bind-Value="newTicket.Title" class="form-control" placeholder="Brief summary of the issue" />
26         </div>
27
28         <div class="mb-3">
29             <label class="form-label">Category</label>
30             <InputSelect @bind-Value="newTicket.CategoryId" class="form-select">
31                 <option value="0">-- Select a Category --</option>
32                 @foreach (var cat in categories)
33                 {
34                     <option value="@cat.Id">@cat.Name</option>
35                 }
36             </InputSelect>
37         </div>
38
39         <div class="mb-3">
40             <label class="form-label">Priority</label>
41             <InputSelect @bind-Value="newTicket.Priority" class="form-select">
42                 @foreach (var priority in Enum.GetValues<TicketPriority>())
43                 {
44                     <option value="@priority">@priority</option>
45                 }
46             </InputSelect>
47         </div>
```

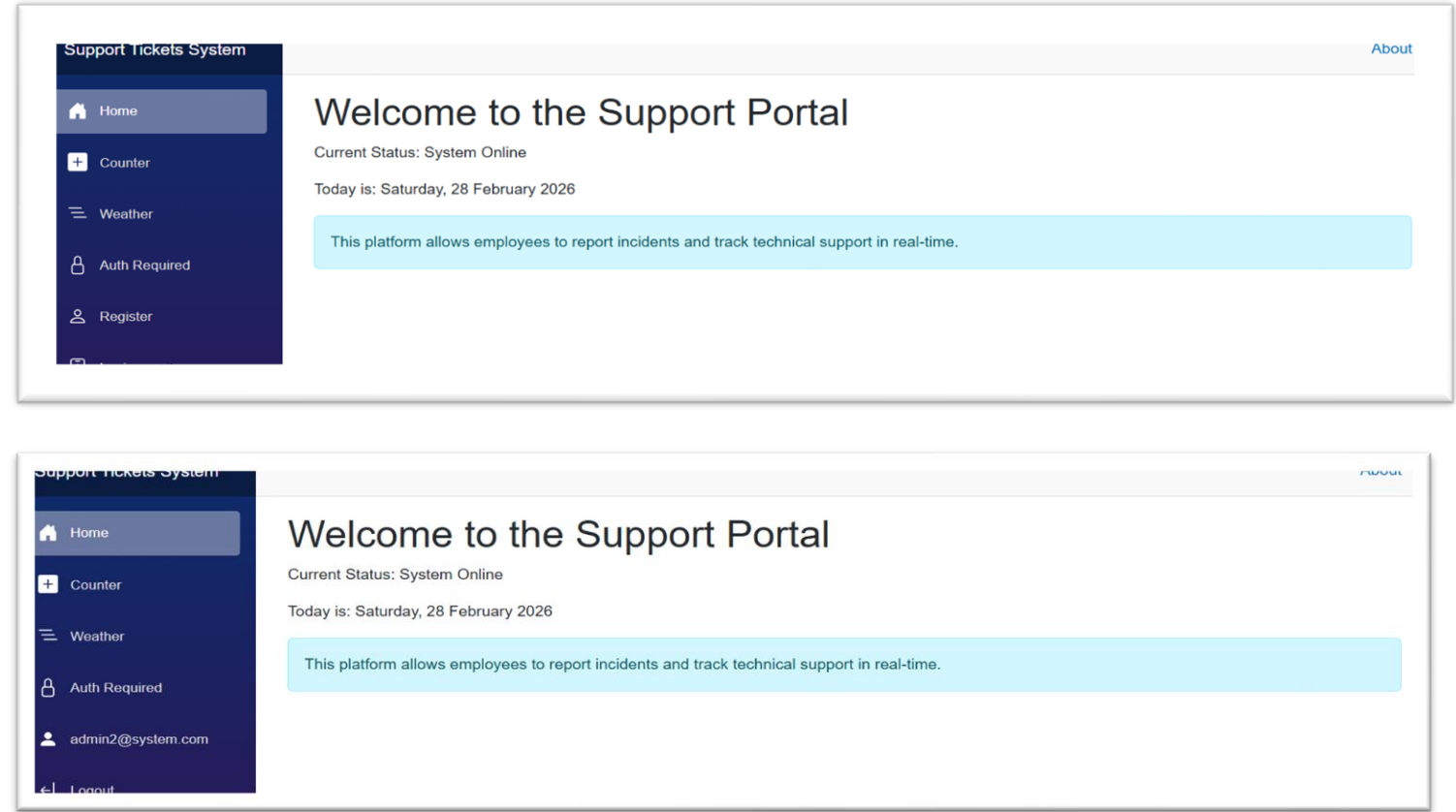
```
48
49
50     <div class="mb-3">
51         <label class="form-label">Description</label>
52         <InputTextArea @bind-Value="newTicket.Description" class="form-control" rows="4" />
53     </div>
54
55     <button type="submit" class="btn btn-primary">Submit Ticket</button>
56     <a href="/" class="btn btn-secondary">Cancel</a>
57 </EditForm>
58 </div>
59
60 @code {
61     private Ticket newTicket = new() { AuthorId = "", Title = "", Description = "" };
62     private List<Category> categories = new();
63
64     protected override async Task OnInitializedAsync()
65     {
66         categories = await TicketService.GetCategoriesAsync();
67
68         // Obtener el ID del usuario actual autenticado
69         var authState = await AuthProvider.GetAuthenticationStateAsync();
70         var user = authState.User;
71         newTicket.AuthorId = user.FindFirst(ClaimTypes.NameIdentifier)?.Value ?? "";
72     }
73
74     private async Task HandleSubmit()
75     {
76         if (newTicket.CategoryId == 0) return; // Validación simple de categoría
77
78         await TicketService.CreateTicketAsync(newTicket);
79         Navigation.NavigateTo("/"); // Por ahora redirigimos al Home
80     }
81 }
```

4. Descripción del código

- 1. **@attribute [Authorize]**: Asegura que nadie pueda entrar a esta página si no ha iniciado sesión.
- 2. **EditForm**: El componente de Blazor que gestiona el estado del formulario y la validación automática.
- 3. **InputSelect con Enums**: Observa cómo iteramos sobre `Enum.GetValues<TicketPriority>()`. Esto es extremadamente eficiente porque si agregas una prioridad nueva en el Módulo 3, el formulario se actualiza solo.
- 4. **AuthenticationStateProvider**: Lo usamos para capturar el `Id` del usuario que está logueado. Este es el `AuthorId` que requiere nuestra base de datos.

Resultado Final del Módulo

- 1. Ejecuta la aplicación e inicia sesión.



2. Navega manualmente a `/tickets/create` (o añade un link en el NavMenu).

The image is a screenshot of a web browser displaying a 'Create New Support Ticket' form. The browser's address bar at the top shows the URL 'https://localhost:7271/tickets/create'. The page layout consists of a dark blue sidebar on the left and a main content area on the right. The sidebar contains navigation links: 'Home' (with a house icon), 'Auth Required' (with a lock icon), 'admin@system.com' (with a user icon), and 'Logout' (with a left arrow icon). The main content area has a header 'Support Tickets System 2020' and an 'About' link. The main heading is 'Create New Support Ticket'. Below this, there are four form fields: 'Title' (containing 'Brief summary of the issue'), 'Category' (a dropdown menu showing '-- Select a Category --'), 'Priority' (a dropdown menu showing 'Medium'), and 'Description' (an empty text area). At the bottom of the form are two buttons: 'Submit Ticket' (in blue) and 'Cancel' (in grey).

3. Llena el formulario y presiona "Submit".

Support Tickets System 2020

Home

Auth Required

admin@system.com

Logout

About

Create New Support Ticket

Title

Error en aplicación de inventario

Category

Software

Priority

High

Description

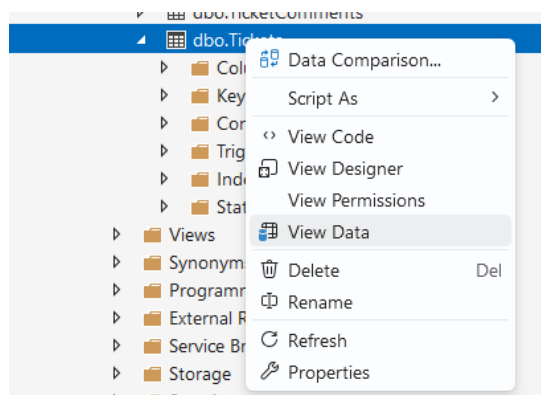
El programado que uso para reportar el inventario físico se queda congelado y no me deja ingresar la información

Submit Ticket

Cancel

4. Crear 3 tickets

5. **Verificación:** Abre SQL Server y consulta la tabla `Tickets`. ¡Deberías ver tu primer registro guardado con el `AuthorId` correcto!

[illegible][illegible]

Checkpoint (Agile)

- [x] Servicio de datos configurado con `IDbContextFactory`.
- [x] Inyección de dependencias funcionando.
- [x] Uso de componentes de formulario (`EditForm`, `InputSelect`).
- [x] Obtención del ID del usuario desde el estado de autenticación.